

Tema 5:

Colas

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Sergio Caverio



Universidad
Rey Juan Carlos

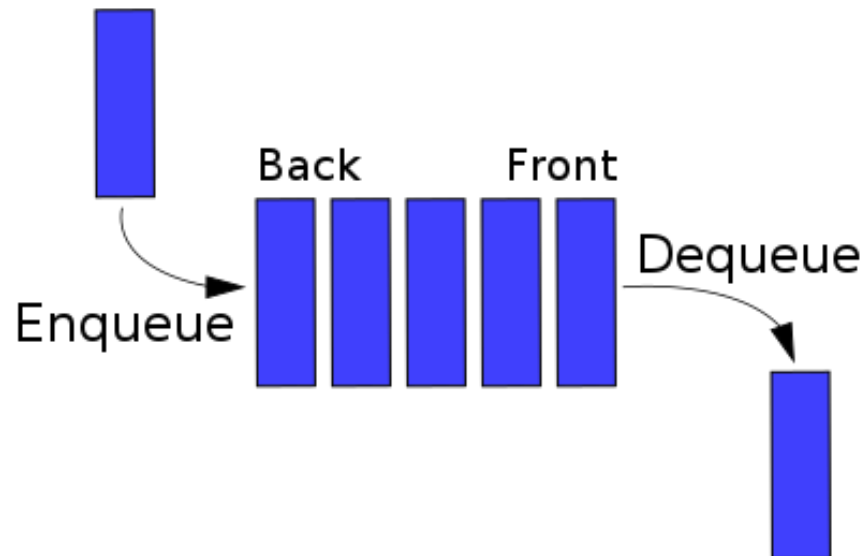
Curso 2025-2026

Contenidos

- Definición
- El TAD Cola: operaciones básicas
- Implementación
- Ejemplos de aplicación
- Colas y genericidad
- Ejercicios propuestos

Cola: definición

- Estructura de datos lineal, dinámica, que da soporte al concepto **FIFO: First In, First Out**

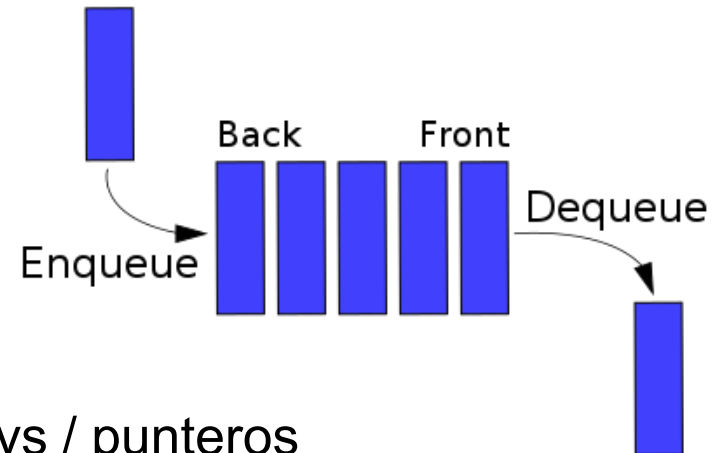


TAD Cola

- Independientemente de su implementación, una cola debe proporcionar al menos 5 operaciones básicas:
 - Meter un elemento (enqueue)
 - Sacar un elemento (dequeue)
 - Consultar si está vacía (empty_queue)
 - Conocer el primer y último elemento (first/last)
 - + Inicializar/crear

TAD Cola: especificación

- **Nombre:** Cola.
- **Valores posibles:** *vacía, otros (según elementos).*
- **Operaciones:**
 - enqueue
 - dequeue
 - first
 - last
 - empty_queue
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)



Implementación

- Dos formas habituales de implementarla: con un array o utilizando memoria dinámica.
 - Externamente no debería ser posible distinguirlo
- Utilizando arrays hay limitaciones de espacio y otros inconvenientes.
- Utilizando punteros y memoria dinámica no hay límite de espacio (en teoría) y la implementación es más eficiente.

Implementación mediante arrays

Implementación mediante arrays

- Necesitaremos dos variables enteras para indicar la posición del primer y último elemento de la cola + un array del tipo base a almacenar



first 1 last 4

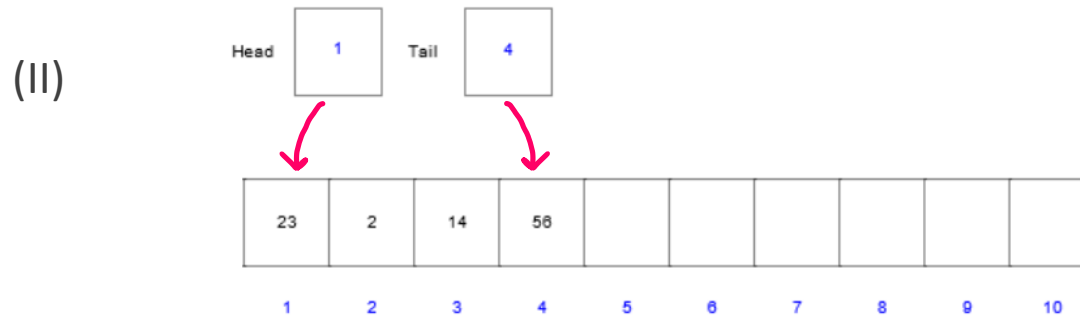
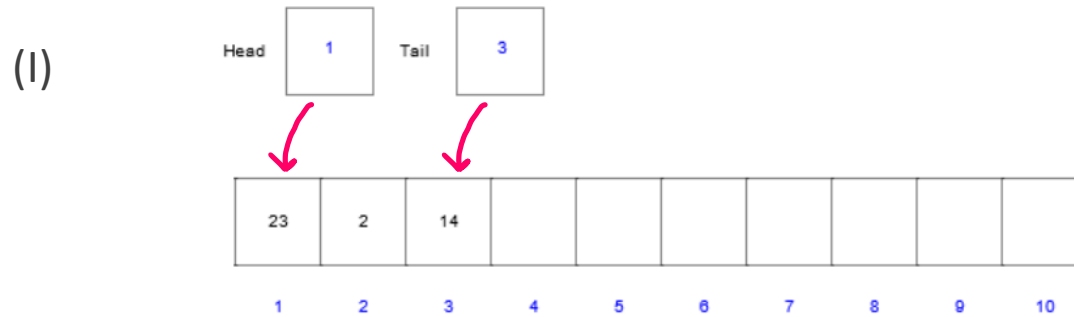


Implementación mediante arrays

- Hay muchas implementaciones posibles, algunas muy sencillas, otras más complicadas, pero todas peores en términos de eficiencia (y a menudo de complejidad de implementación) que las implementaciones con punteros.
 - a) Mediante un array lineal con eliminación simple
 - b) Mediante un array lineal con eliminación reorganizada
 - c) Mediante un array circular con cuenta de longitud
 - d) Mediante un array circular con posición libre
 - e) Etc.

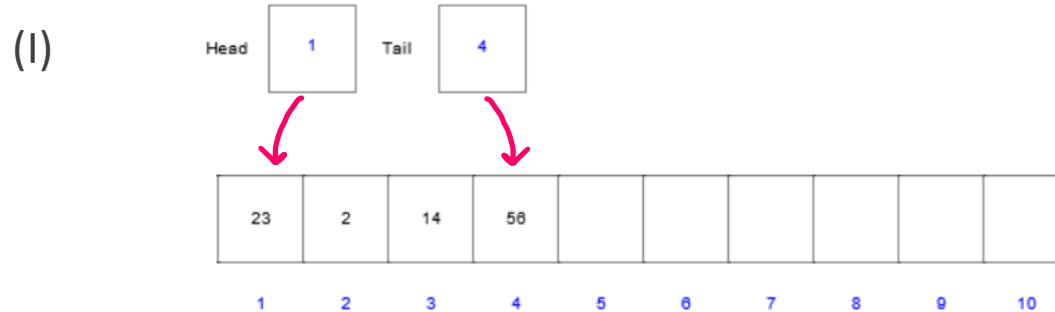
(a) Array lineal con eliminación simple

- La inserción se produce siempre en el primer hueco disponible (por la parte de atrás):



(a) Array lineal con eliminación simple

- Sacar un elemento consiste en mover el índice del primer elemento al siguiente ($\text{first} := \text{first} + 1$):

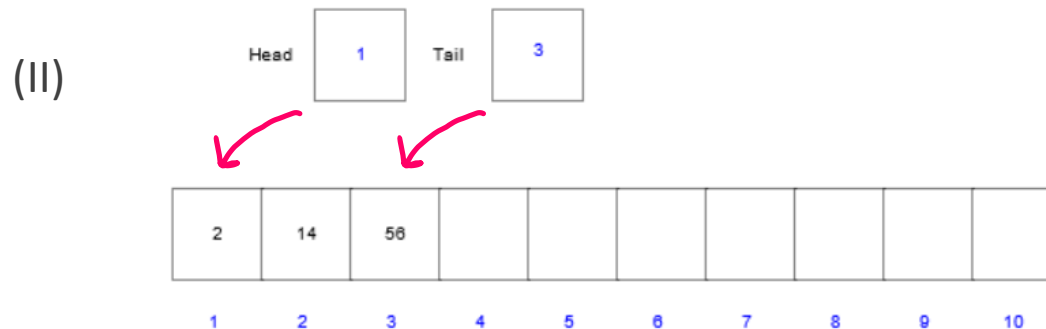
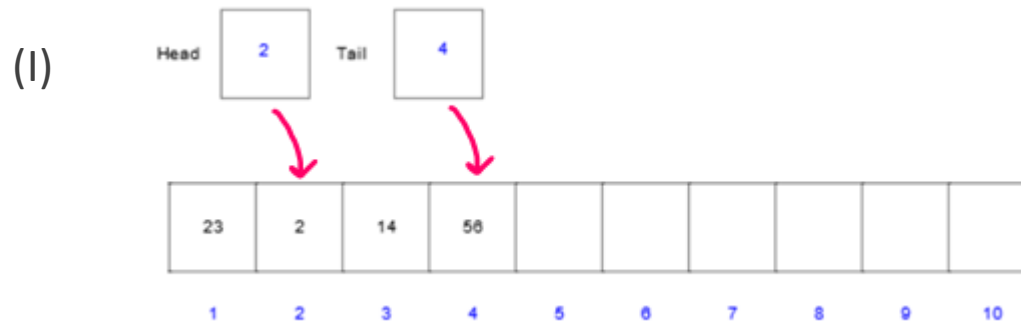


(a) Array lineal con eliminación simple

- Esta implementación es muy sencilla, pero...
- La cola tiene un tamaño prefijado (rígido)
- Sacar un elemento es muy sencillo y eficiente ($O(1)$) pero deja memoria no utilizable lo cual implica:
 - La necesidad de reorganizarla en algún momento
 - El riesgo de no hacerlo y quedarse sin espacio cuando realmente sí lo hay

(b) Array lineal con reorganización

- Al sacar un elemento de la cola se reorganizan los elementos del array, situando el primero en la posición 1, etc.



(b) Array lineal con reorganización

- La implementación incluye una operación de borrado no tan simple de implementar, si bien no hace falta “front”:

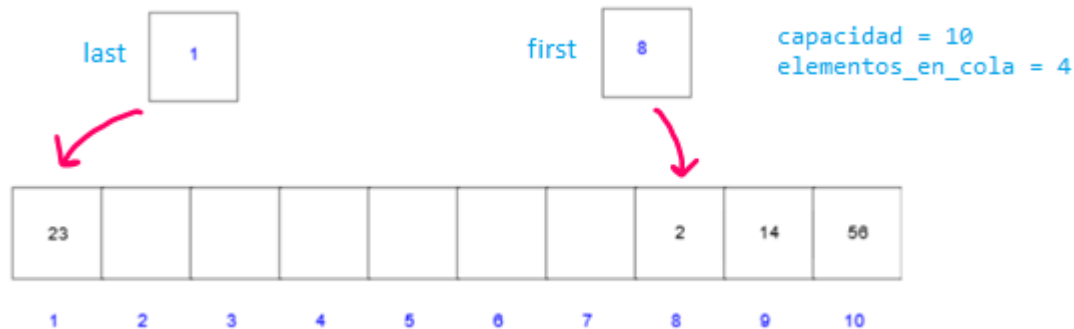
```
if (not empty_queue(cola)) then
  for i:=1 to tail-1
    cola[i] = cola[i + 1];
  tail:= tail-1
```

→ $O(n)$

- La complejidad del borrado pasa a ser $O(n)$ frente a la complejidad constante de la implementación (a)

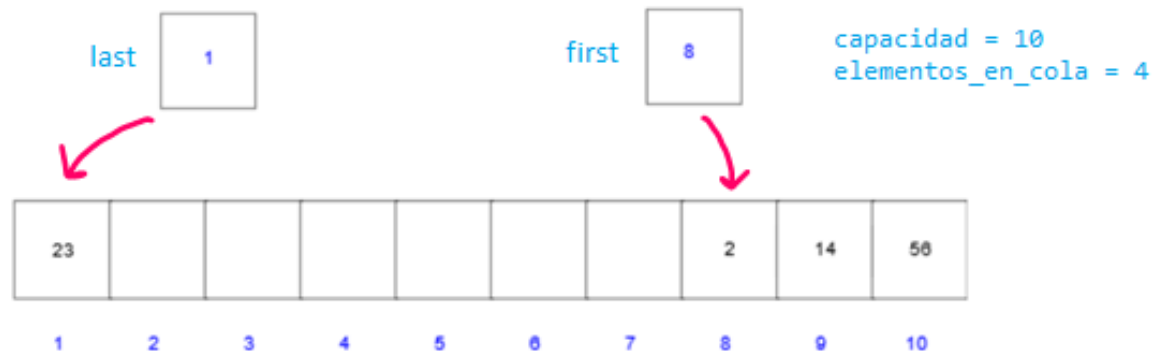
(c) Array circular

- La inserción es simple: avanzar el índice `last` a la siguiente posición y almacenar el dato: $\text{new_last} = (\text{last} \bmod \text{TAM}) + 1$
- La eliminación no reorganiza: simplemente se avanza `first` a la siguiente posición: $\text{new_first} = (\text{first} \bmod \text{TAM}) + 1$
- El array se diseña para continuar insertando en el hueco que queda entre `first` y `last`



(c) Array circular

- Para saber si la cola está vacía la implementación más sencilla es un entero `elementos_en_cola`: si cero $\rightarrow$ vacía.
- Cola llena: si `elementos_en_cola` es igual a la capacidad.
- Otra implementación deja un elemento vacío en el array para separar los índices `last` y `first`.



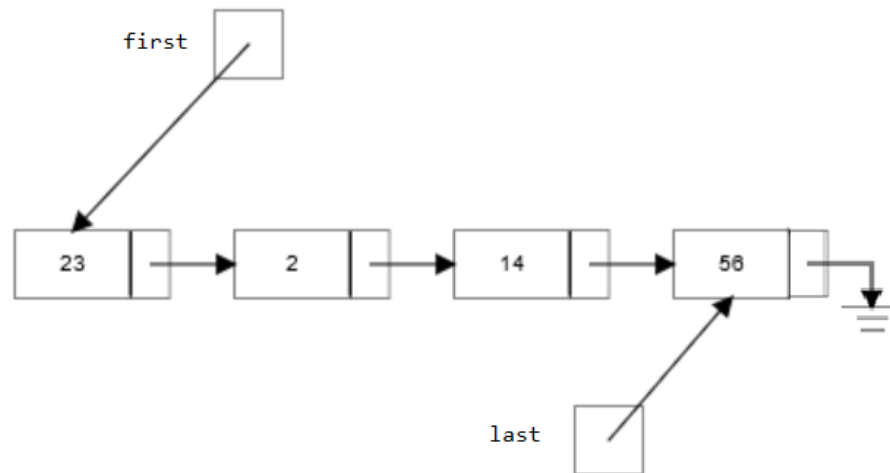
Cola como array (resumen)

- **Desventajas:**
 - Es necesario conocer el tamaño a priori.
 - La eliminación es poco eficiente en implementación lineal, no en el array circular
 - La implementación no es tan inmediata como en la pila
- Es necesario establecer **precondiciones** en varias operaciones cuya implementación más lógica sería utilizar excepciones
 - Es necesaria una reflexión diferente para cada tipo base.

Implementación en memoria dinámica

Implementación con memoria dinámica

- Utilizamos una estructura con dos punteros: `first` y `last`
- No hay máximo de elementos
- Inicialmente `first` y `last` están a NIL
- Antes de eliminar un elemento hay que comprobar si es posible
- La inserción y borrado son muy eficientes: complejidad $O(1)$

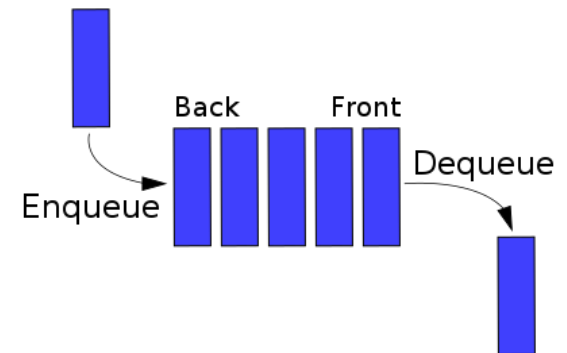


Implementación con memoria dinámica

```
unit uColaInteger;  
interface  
  
type  
    // Estructura del nodo  
    nodo = record  
        info: integer;  
        sig: ^nodo;  
    end;  
    // Estructura de la cola  
    tCola = record  
        first, last: ^nodo;  
    end;
```

Implementación con memoria dinámica

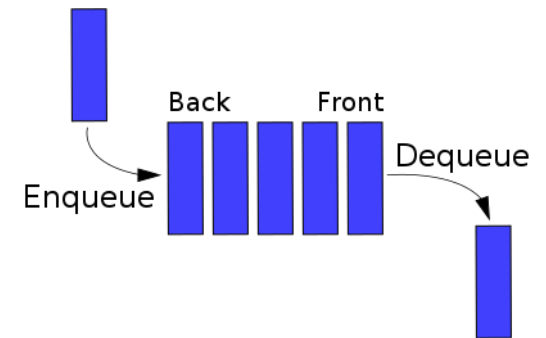
```
procedure initialize_queue(var c: tCola);  
function empty_queue(c: tCola): boolean;  
function first(c: tCola): integer;  
function last(c: tCola): integer;  
procedure enqueue(var c: tCola; x: integer);  
procedure dequeue(var c: tCola);
```



Implementación con memoria dinámica

```
// Inicializar la cola
procedure initialize_queue(var c: tCola);
begin
    c.first := nil;
    c.last := nil;
end;

// Verificar si la cola está vacía
function empty_queue(c: tCola): boolean;
begin
    empty_queue := c.first = nil;
end;
```



Implementación con memoria dinámica

// Obtener el primer elemento de la cola

```
function first(c: tCola): integer;
```

```
begin
```

```
    if not empty_queue(c) then
```

```
        first := c.first^.info
```

```
end;
```

// Obtener el último elemento de la cola

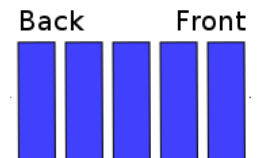
```
function last(c: tCola): integer;
```

```
begin
```

```
    if not empty_queue(c) then
```

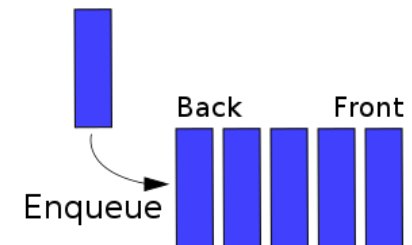
```
        last := c.last^.info
```

```
end;
```



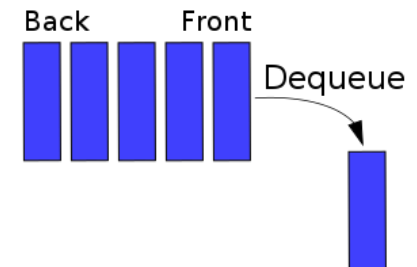
Implementación memoria dinámica

```
// Agregar un elemento a la cola
procedure enqueue(var c: tCola; x: integer);
var
    nuevo: ^nodo;
begin
    new(nuevo);
    nuevo^.info := x;
    nuevo^.sig := nil;
    if empty_queue(c) then
        c.first := nuevo
    else
        c.last^.sig := nuevo;
    c.last := nuevo;
end;
```



Implementación memoria dinámica

```
// Eliminar un elemento de la cola
procedure dequeue(var c: tCola);
var
    aux: ^nodo;
begin
    if not empty_queue(c) then
    begin
        aux := c.first;
        c.first := c.first^.sig;
        dispose(aux);
        if c.first = nil then
            c.last := nil;
        end;
    end;
end;
```



Funcionalidades adicionales

- Podemos ampliar nuestro TAD Cola con algunas operaciones extra tales como:
 - `clear()` → Vacía la cola
 - `toString()` → representación mediante una cadena de los elementos de la cola
 - `num_elems()` → Cuenta el número de elementos de la cola
 - `copy()` → Genera un duplicado de la cola

Funcionalidades adicionales

```
// Elimina todos los elementos (limpia) la cola  
procedure clear(var c: tCola);  
begin  
    while not empty_queue(c) do  
        dequeue(c);  
end;
```

Funcionalidades adicionales

```
// Convertir la cola a una cadena de caracteres
function toString(c: tCola): string;
var
    aux: ^nodo;
    s: string;
begin
    aux := c.first;
    s := '[';
    while aux <> nil do begin
        s := s + IntToStr(aux^.info);
        aux := aux^.sig;
        if aux <> nil then
            s := s + ', ';
    end;
    s := s + ']';
    toString := s;
end;
```

Funcionalidades adicionales

```
// Contar el número de elementos de la cola
function num_elems(c: tCola): integer;
var
    aux: ^nodo;
    count: integer;
begin
    aux := c.first;
    count := 0;
    while aux <> nil do begin
        count := count + 1;
        aux := aux^.sig;
    end;
    num_elems := count;
end;
```

¿Cómo podríamos implementar esta funcionalidad en $O(1)$?

Funcionalidades adicionales

```
// Copiar una cola
procedure copy(c : tCola; var c2: tCola);
var
    aux: ^nodo;
begin
    initialize_queue(c2);
    aux := c.first;
    while aux <> nil do begin
        enqueue(c2, aux^.info);
        aux := aux^.sig;
    end;
end;
```

Aplicaciones

- Muy utilizadas en computación para un gran número de aplicaciones tales como:
 - Gestión de tareas de la CPU
 - Organización de trabajos de impresión
 - Gestión de dispositivos y recursos compartidos
 - Gestión de centros de llamadas
 - Almacenamiento en búfer
 - Etc.

Complejidad

- Todas las operaciones básicas de las colas son de complejidad $O(1)$
 - `initialize_queue`
 - `enqueue`
 - `dequeue`
 - `first/last`
 - `empty_queue`
- Todas las operaciones aquí incluidas como adicionales son dependientes del número de elementos $\rightarrow O(n)$
 - `to_string`
 - `clear`
 - `copy_queue`
 - `num_elems_queue`